Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce Department of Information Technology (B.Sc.I.T Semester V)
APPLIED BUSINESS ANALYTICS PRACTICAL

Practical – 3A&3B

Roll No.:T010	Name: Sagar Kandari
Class: TYIT	Batch:1
Date of Assignment:14-08-26	Date/Time of Submission:14-08-26

Q1.a. Use a dataset with multiple variables and perform correlation analysis to determine the strength and direction of relationships between pairs of variables.

Code-:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```
[1] ✓ 50.4s Python

```
df = pd.read_csv("housing - housing.csv")
```
[2] ✓ 0.0s Python

```
print(df.head())
```
[3] ✓ 0.0s Python

```
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23     37.88                  41          880           129.0
1    -122.22     37.86                  21         7099          1106.0
2    -122.24     37.85                  52         1467           190.0
3    -122.25     37.85                  52         1274           235.0
4    -122.25     37.85                  52         1627           280.0

   population  households  median_income  median_house_value ocean_proximity
0         322         126         8.3252              452600        NEAR BAY
1        2401        1138         8.3014              358500        NEAR BAY
2         496         177         7.2574              352100        NEAR BAY
3         558         219         5.6431              341300        NEAR BAY
4         565         259         3.8462              342200        NEAR BAY
```

```
print(df.select_dtypes(include="number").columns)
```

[4] ✓ 0.0s Python

```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
       'total_bedrooms', 'population', 'households', 'median_income',
       'median_house_value'],
      dtype='str')
```

```
correlation = df.corr(numeric_only=True)
print(correlation)
```

[5] ✓ 0.0s Python

```
                    longitude  latitude  housing_median_age  total_rooms  \
longitude            1.000000 -0.924664           -0.108197     0.044568
latitude            -0.924664  1.000000            0.011173    -0.036100
housing_median_age  -0.108197  0.011173            1.000000    -0.361262
total_rooms          0.044568 -0.036100           -0.361262     1.000000
total_bedrooms       0.069608 -0.066983           -0.320451     0.930380
population           0.099773 -0.108785           -0.296244     0.857126
households           0.055310 -0.071035           -0.302916     0.918484
median_income       -0.015176 -0.079809           -0.119034     0.198050
median_house_value  -0.045967 -0.144160            0.105623     0.134153

                    total_bedrooms  population  households  median_income  \
longitude                 0.069608    0.099773    0.055310      -0.015176
latitude                 -0.066983   -0.108785   -0.071035      -0.079809
housing_median_age       -0.320451   -0.296244   -0.302916      -0.119034
total_rooms               0.930380    0.857126    0.918484       0.198050
total_bedrooms            1.000000    0.877747    0.979728      -0.007723
```

```
                    median_house_value
longitude                    -0.045967
latitude                     -0.144160
...
population                   -0.024650
households                    0.065843
median_income                 0.688075
median_house_value            1.000000
```

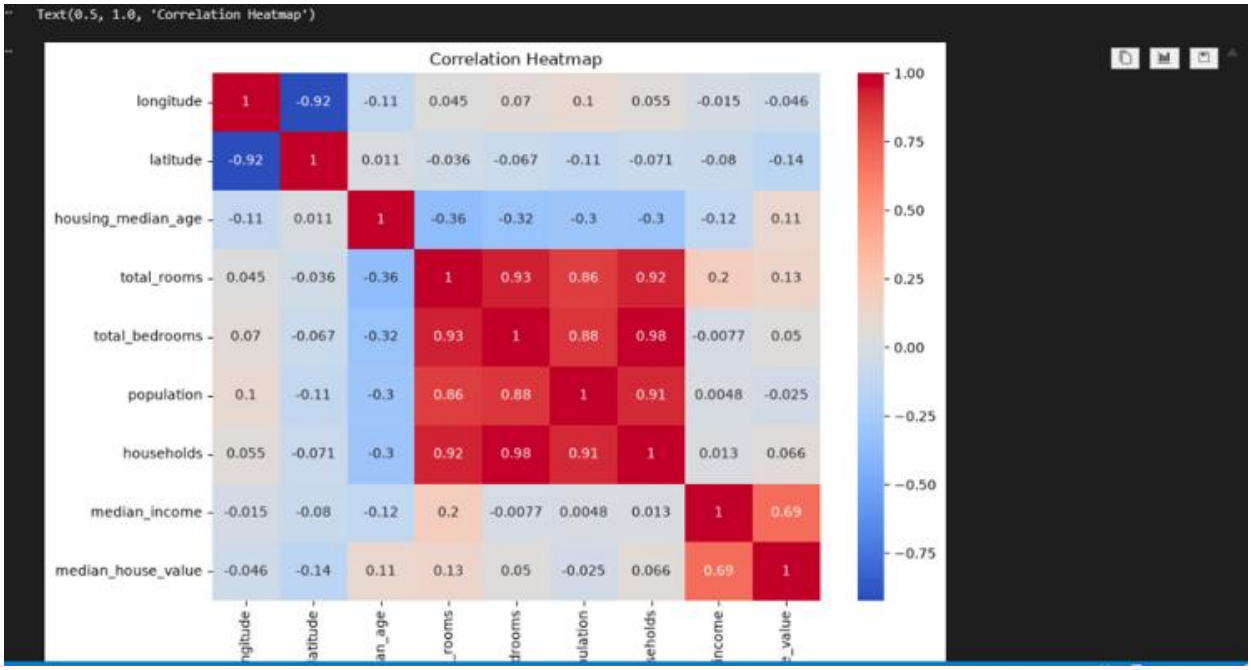
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```
plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")
```

[6] ✓ 1.1s Python

```
Text(0.5, 1.0, 'Correlation Heatmap')
```

Sagar Kandari
T010



Sagar Kandari
T010

Practical 3B

Apply regression analysis to identify the relationship between an independent variable (e.g., advertising expenditure) and a dependent variable (e.g., sales revenue).

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

✓ 1.7s Python

```
X = df[["median_income"]]
y = df["median_house_value"]
```

✓ 0.0s Python

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

✓ 0.0s Python

```
model = LinearRegression()
```

✓ 0.0s Python

```
model.fit(X_train, y_train)
```

✓ 0.1s Python

▸ LinearRegression ⓘ ⓘ

```
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
```

[12] ✓ 0.0s Python

```
Coefficient: 41933.84939381274
Intercept: 44459.729169078666
```

```
y_pred = model.predict(X_test)
```

[13] ✓ 0.0s Python

```
print("Predicted Values:")

print(y_pred[:10])
```

[14] ✓ 0.0s Python

```
Predicted Values:
[114958.91676996 150606.88213964 190393.71844449 285059.38345102
 200663.31816103 242165.24890609 257647.22610228 199229.18051176
 245893.1681172  384677.43607096]
```

```
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)
```

[15] ✓ 0.0s Python

```
Mean Squared Error: 7091157771.76555
```

```
r2 = r2_score(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)

print("R2 Score:", r2)
```

✓ 0.0s

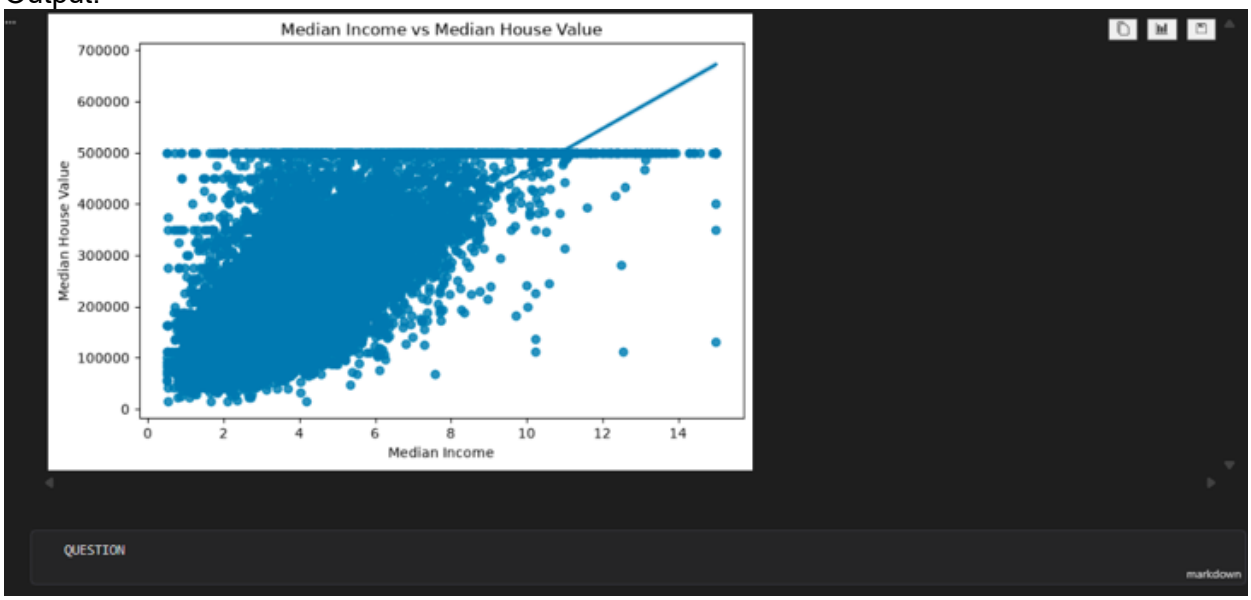
```
R2 Score: 0.4588591890384665 6
```

```
plt.figure(figsize=(8, 5))

sns.regplot(
    x="median_income",
    y="median_house_value",
    data=df
)

plt.xlabel("Median Income")

plt.ylabel("Median House Value")

plt.title("Median Income vs Median House Value")

plt.show()
```

✓ 0.9s

Sagar Kandari
T010

Output:-



Sagar Kandari
T010

QUESTION -: 3A

Q.1 Calculate the complete correlation matrix for all numerical variables.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

✓ 0.0s

```
print("\nQ.1 COMPLETE CORRELATION MATRIX")
print("=" * 60)

# Select numerical columns
numeric_df = df.select_dtypes(include="number")

# Correlation matrix
correlation_matrix = numeric_df.corr()

print(correlation_matrix.round(4))

# Make a copy
corr = correlation_matrix.copy()

# Remove diagonal using pandas instead of np.fill_diagonal()
for column in corr.columns:
    corr.loc[column, column] = None

# Strongest positive correlation
positive_pair = corr.stack().idxmax()
positive_value = corr.stack().max()

# Strongest negative correlation
negative_pair = corr.stack().idxmin()
negative_value = corr.stack().min()

# Weakest correlation
weakest_pair = corr.abs().stack().idxmin()
weakest_value = corr.loc[weakest_pair[0], weakest_pair[1]]

print("\nStrongest Positive Correlation:")
print(positive_pair, "=", round(positive_value, 4))

print("\nStrongest Negative Correlation:")
print(negative_pair, "=", round(negative_value, 4))

print("\nWeakest Correlation:")
print(weakest_pair, "=", round(weakest_value, 4))
```

✓ 0.0s Python

```
Q.1 COMPLETE CORRELATION MATRIX
============================================================
                longitude  latitude  housing_median_age  total_rooms  \
```

```
...
    Q.1 COMPLETE CORRELATION MATRIX
    ============================================================
                        longitude  latitude  housing_median_age  total_rooms  \
    longitude              1.0000   -0.9247             -0.1082       0.0446
    latitude              -0.9247    1.0000              0.0112      -0.0361
    housing_median_age    -0.1082    0.0112              1.0000      -0.3613
    total_rooms            0.0446   -0.0361             -0.3613       1.0000
    total_bedrooms         0.0696   -0.0670             -0.3205       0.9304
    population             0.0998   -0.1088             -0.2962       0.8571
    households             0.0553   -0.0710             -0.3029       0.9185
    median_income         -0.0152   -0.0798             -0.1190       0.1980
    median_house_value    -0.0460   -0.1442              0.1056       0.1342

                        total_bedrooms  population  households  median_income  \
    longitude                   0.0696      0.0998      0.0553        -0.0152
    latitude                   -0.0670     -0.1088     -0.0710        -0.0798
    housing_median_age         -0.3205     -0.2962     -0.3029        -0.1190
    total_rooms                 0.9304      0.8571      0.9185         0.1980
    total_bedrooms              1.0000      0.8777      0.9797        -0.0077
    population                  0.8777      1.0000      0.9072         0.0048
    households                  0.9797      0.9072      1.0000         0.0130
    median_income              -0.0077      0.0048      0.0130         1.0000
    median_house_value          0.0497     -0.0246      0.0658         0.6881
    ...
    ('longitude', 'latitude') = -0.9247

    Weakest Correlation:
    ('population', 'median_income') = 0.0048
    Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Q.2 Build a regression model using:

```python
print("\n\nQ.2 MULTIPLE REGRESSION")
print("------------------------------")

# X variables
X = df[
    [
        "median_income",
        "housing_median_age",
        "total_rooms",
        "population",
        "households"
    ]
]

# Y variable
y = df["median_house_value"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
model = LinearRegression()

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
r2_multiple = r2_score(y_test, y_pred)
mse_multiple = mean_squared_error(y_test, y_pred)
```

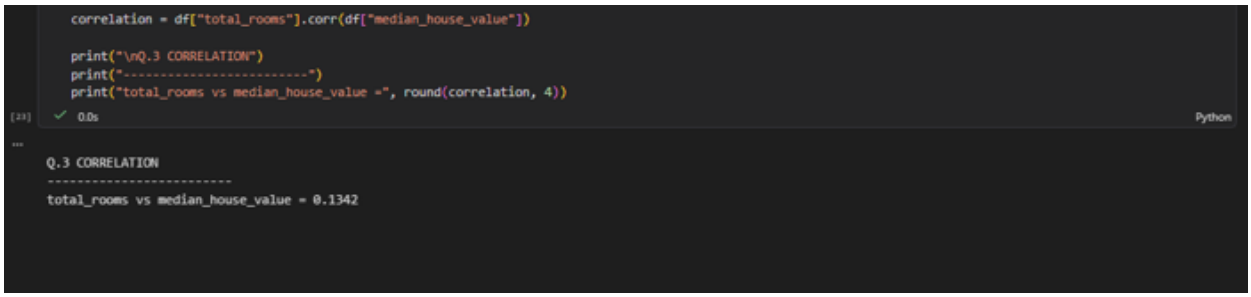
0.0s Python

```
print("\n\nSIMPLE REGRESSION")
print("--------------------------------")

X_simple = df[["median_income"]]

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_simple, y, test_size=0.2, random_state=42
)

simple_model = LinearRegression()

simple_model.fit(X_train_s, y_train_s)

y_pred_simple = simple_model.predict(X_test_s)

r2_simple = r2_score(y_test_s, y_pred_simple)
mse_simple = mean_squared_error(y_test_s, y_pred_simple)

print("Simple Regression R2:", round(r2_simple, 4))
print("Simple Regression MSE:", round(mse_simple, 2))


# ================================================
# COMPARISON
# ================================================

print("\n\nMODEL COMPARISON")
print("--------------------------------")

comparison = pd.DataFrame({
    "Model": [
        "Simple Regression",
        "Multiple Regression"
    ],
    "R2": [
        r2_simple,
        r2_multiple
    ],
    "Mean Squared Error": [
        mse_simple,
        mse_multiple
    ]
})

print(comparison.round(4))
```

0.0s Python

```
2 MULTIPLE REGRESSION
--------------------------------
ltiple Regression R2: 0.549
ltiple Regression MSE: 5909928044.7


MPLE REGRESSION
--------------------------------
mple Regression R2: 0.4589
mple Regression MSE: 7091157771.77


DEL COMPARISON
--------------------------------
                Model      R2  Mean Squared Error
    Simple Regression  0.4589        7.091158e+09
  Multiple Regression  0.5490        5.909928e+09
```

Q.3 Find the correlation between:



```
correlation = df["total_rooms"].corr(df["median_house_value"])

print("\nQ.3 CORRELATION")
print("------------------------")
print("total_rooms vs median_house_value =", round(correlation, 4))
```

[23] ✓ 0.0s Python

```
Q.3 CORRELATION
------------------------
total_rooms vs median_house_value = 0.1342
```